Project Report: AI Writer · Advanced Text Summarization

Author: Abhijit
Project Date: 2026-02-26

---

1. Abstract

The AI Writer is a modern, responsive web application designed to provide abstractive text summarization. By leveraging the Hugging Face Inference API and state-of-the-art transformer models (BART and DistilBART), the application empowers users to summarize lengthy texts and PDF documents quickly and accurately. The goal of this project is to save users time while reading long articles, reports, or research papers by providing concise, comprehensible summaries alongside keyword extraction and readability metrics.

2. Introduction

In the digital age, the volume of text data generated daily is overwhelming. Professionals, students, and researchers often lack the time to read entire documents. To address this, automated text summarization has emerged as a crucial Natural Language Processing (NLP) task.

This project aims to build a scalable and user-friendly web interface that abstracts away the complexity of NLP models. Instead of running heavy machine learning models locally·which demands significant computational power·the application connects to cloud-hosted models, ensuring high performance, low latency, and broad accessibility.

3. System Features

- **Dual-Model Deep Learning Architecture**: Offers a "Standard" (fast) mode utilizing `distilbart-cnn-12-6` and a "Pro" (advanced) mode utilizing `bart-large-cnn`.
- **Dynamic Summary Lengths**: Users can customize the generated summary length (Short, Medium, Detailed).
- **Multi-Format Support**: Users can either paste plain text directly or upload a `.pdf` file.
- **Intelligent Keyword Extraction**: Employs TF-IDF (Term Frequency-Inverse Document Frequency) algorithm to extract top keywords representing the core theme of the document.
- **Readability and Compression Metrics**: Calculates the Flesch Reading Ease score to determine how understandable the summary is. It also compares the input word count to the output word count to showcase the compression ratio.
- **Integrated Caching**: Implements backend caching for frequent identical requests to minimize redundant API calls.

4. System Architecture

The platform follows a standard client-server architecture: **1. Frontend (Client)**: Built with HTML5, CSS3, JavaScript, and Bootstrap 5. It manages the user interface, form validation, and displays the summary, keywords, and metrics dynamically without page reloads. **2. Backend (Server)**: A Python Flask server that handles file uploads, text cleaning, text chunking, and API communication. **3. Inference Engine**: The Hugging Face Inference API (`router.huggingface.co/hf-inference/models/`) which hosts the pre-trained NLP models.

5. Implementation Details

5.1 Libraries and Frameworks

- **Flask**: Lightweight WSGI web application framework.

- **pdfplumber**: Used for precise text extraction from uploaded PDF documents.
- **Requests**: Manages HTTP requests to the Hugging Face API.
- **scikit-learn**: Provides the `TfidfVectorizer` for term frequency analysis.
- **textstat**: Calculates the Flesch Reading Ease readability metric.

5.2 Text Chunking

Transformer models have strict input token limits (often 512 or 1024 tokens). To summarize longer documents, the application splits the input text into manageable chunks. Each chunk is summarized independently, and the resulting mini-summaries are concatenated.

5.3 Error Handling

Robust error handling has been integrated into both frontend and backend. Failed API calls trigger visually clear HTML alerts instead of silent console errors.

6. Results and Evaluation

During testing, the `distilbart-cnn-12-6` model demonstrated highly efficient inference speeds suitable for real-time web usage. The `bart-large-cnn` model provided structurally superior and more coherent summaries for complex inputs. For a standard 100-word technical paragraph, the system successfully compressed the text by over 20% while maintaining a Flesch readability score above 25 (indicating technical or college-level material), preserving the original context perfectly.

7. Configuration and Security

The system secures its cloud communication by injecting the designated Hugging Face Auth Token as an Environment Variable (`HF_API_TOKEN`). This prevents hardcoding sensitive credentials in the source code.

8. Conclusion

The AI Writer successfully demonstrates the integration of modern cloud-hosted NLP models within a standard web framework. It provides a highly applicable tool for qualitative text compression, readability analysis, and keyword extraction, operating stably through caching and dynamic text-chunking algorithms.

9. Future Enhancements

- Incorporate user authentication to save summary histories.
- Add support for more file formats (e.g., `.docx`, `.epub`).
- Introduce multilingual summarization.
- Implement progressive UI loading for very large documents.